

ESCUELA MILITAR DE INGENIERÍA MCAL. ANTONIO

JOSÉ DE SUCRE

“BOLIVIA”



CRIPTOGRAFIA MODERNA

ESTUDIANTES:

- Genaro Denis Huanca Vargas
- Mishell Diaz Patty
- Dennis Omar Paz Aranibar
- Elmer Alcides Apaza Mamani
- Jose Antonio Pachas Chipana
- Denis Patzi Canaviri
- Jorge Andres Cespedes Calderon
- Luz Julieta Tola Quispe
- Erik Leonardo Morales Escalante
- Rene Israel Apaza Laura
- Saul Armin Marca Cruz

DOCENTE:

La Paz – Bolivia

2026

INTRODUCCIÓN

La **criptografía moderna** constituye uno de los pilares fundamentales de la ciberseguridad en la era digital. Su evolución histórica la ha transformado de un arte basado en la sustitución lingüística en una disciplina científica rigurosa, sustentada en la **complejidad algorítmica** y principios matemáticos avanzados. A diferencia de los métodos clásicos, la seguridad contemporánea se fundamenta en el **Principio de Kerckhoffs**, el cual establece que la robustez de un sistema reside exclusivamente en la protección de las claves, permitiendo que los algoritmos sean públicos y auditables mientras permanecen computacionalmente intratables.

Este informe técnico analiza las dos vertientes principales de la materia: la **criptografía simétrica**, caracterizada por su alta eficiencia en el procesamiento de grandes volúmenes de información mediante el uso de una clave compartida, y la **criptografía asimétrica**, que revolucionó la comunicación digital al introducir pares de claves (**pública** y **privada**), resolviendo así el problema de la distribución de secretos en canales inseguros.

A lo largo del documento, se examinan algoritmos simétricos de alto rendimiento como **Salsa20**, **ChaCha20** y **AES**, junto con sistemas asimétricos fundamentales como **RSA**, **ElGamal**, **Rabin** y el criptosistema de **Paillier**. El análisis abarca tanto los fundamentos matemáticos subyacentes —incluyendo la **aritmética modular**, la factorización de enteros y el problema del logaritmo discreto— como sus aplicaciones prácticas en protocolos de comunicación segura tales como **TLS** y **VPN**, evaluando su resistencia frente a ataques actuales y los desafíos emergentes de la **computación cuántica**.

CRIPTOGRAFÍA SIMÉTRICA

1. ChaCha20

1.1 Introducción

El algoritmo ChaCha20 es un método moderno de cifrado perteneciente a la criptografía simétrica. Fue diseñado por el criptógrafo Daniel J. Bernstein como una mejora del algoritmo Salsa20, con el objetivo de ofrecer mayor seguridad, rapidez y eficiencia en dispositivos modernos.

ChaCha20 se utiliza actualmente en múltiples sistemas de seguridad informática debido

a su alto rendimiento en software y su resistencia frente a ataques criptográficos conocidos. Es ampliamente empleado en protocolos de comunicación segura como TLS, VPN y aplicaciones de mensajería cifrada.

A diferencia de los cifrados por bloques tradicionales, ChaCha20 funciona como un cifrado de flujo, generando una secuencia pseudoaleatoria llamada *keystream*, la cual se combina con el mensaje original para producir el texto cifrado.

1.2 Funcionamiento del Algoritmo

ChaCha20 cifra la información mediante la generación de un flujo de datos pseudoaleatorio basado en tres elementos principales:

- una clave secreta compartida,
- un nonce único,
- y un contador de bloques.

El algoritmo no cifra directamente el mensaje. Primero genera un keystream mediante operaciones matemáticas internas y luego aplica una operación XOR entre el keystream y el mensaje original.

El proceso general es el siguiente:

1. Se convierte el mensaje en valores numéricos.
2. Se construye un estado interno usando constantes, clave, contador y nonce.
3. Se aplican 20 rondas de operaciones matemáticas.
4. Se genera el keystream de 64 bytes.
5. Se realiza la operación XOR entre el keystream y el mensaje.
6. Se obtiene el texto cifrado.

Para descifrar, se repite el mismo proceso utilizando los mismos parámetros.

1.3 Partes del Algoritmo

1.3.1. Clave Secreta (Key)

Es un valor de 256 bits compartido entre emisor y receptor.

Funciones principales:

- controla la generación del keystream,
- garantiza la confidencialidad,
- debe mantenerse secreta.

Si cambia la clave, el resultado cifrado cambia completamente.

1.3.2. Nonce

El nonce (*Number Used Once*) es un número único asociado a cada mensaje cifrado.

Características:

- no necesita ser secreto,
- debe ser diferente para cada cifrado usando la misma clave,
- evita repetir el mismo keystream.

Su reutilización con la misma clave compromete la seguridad del sistema.

1.3.3. Contador (Block Counter)

Indica el número de bloque del mensaje que está siendo cifrado.

ChaCha20 procesa datos en bloques de: 64 bytes

Cada bloque incrementa el contador:

- Bloque 1 → contador 1
- Bloque 2 → contador 2
- Bloque 3 → contador 3

Esto permite cifrar mensajes largos de forma continua.

1.3.4. Constantes

Son valores fijos definidos dentro del algoritmo.

Funciones:

- inicializar el estado interno,
- evitar manipulaciones del sistema,
- asegurar estructura criptográfica estable.

No cambian durante la ejecución.

1.3.5. Función Quarter Round

Es la operación matemática fundamental del algoritmo.

Realiza:

- suma modular,
- operación XOR,
- rotación de bits.

Estas operaciones mezclan los datos repetidamente para producir números impredecibles.

ChaCha20 ejecuta 20 rondas de estas transformaciones.

1.3.6. Keystream

Es una secuencia pseudoaleatoria generada por el algoritmo.

Características:

- tamaño de 64 bytes por bloque,
- depende de clave + nonce + contador,
- nunca debe repetirse con la misma clave.

El keystream se combina con el mensaje mediante XOR para obtener el texto cifrado.

1.4. Ventajas de ChaCha20

ChaCha20 ofrece varias ventajas que lo hacen una elección popular para comunicaciones seguras:

1. **Velocidad:** ChaCha20 es conocido por su velocidad y eficiencia. Está diseñado para ser altamente performante y puede implementarse en diversas plataformas y dispositivos sin sacrificar la seguridad.
2. **Resistencia a Ataques de Tiempo y de Caché:** ChaCha20 está diseñado para mitigar ataques de tiempo y de caché. Estos ataques buscan explotar las variaciones en el tiempo que toman diferentes operaciones criptográficas, pero el diseño de ChaCha20 impide que los atacantes obtengan información a través del análisis del tiempo.
3. **Propiedades de Seguridad Fuertes:** ChaCha20 está diseñado para proporcionar altos estándares de seguridad. Muestra una resistencia significativa contra ataques criptográficos conocidos, como los ataques diferenciales y las aproximaciones lineales.
4. **Flexibilidad:** ChaCha20 puede ser implementado y adaptado fácilmente para diferentes tipos de aplicaciones y plataformas, lo que lo hace una elección versátil para protocolos de comunicación seguros.

1.5. Consejos de Prevención

Para asegurar la transmisión segura de datos y proteger contra ataques criptográficos al usar ChaCha20, considere los siguientes consejos de prevención:

1. **Implementar ChaCha20 en los protocolos criptográficos:** Integre ChaCha20 en sus protocolos criptográficos para garantizar la transmisión segura de datos y proteger la información sensible del acceso no autorizado.
2. **Elegir claves, nonces y contadores fuertes y únicos:** Use claves largas y aleatorias, nonces únicos y contadores para evitar que los atacantes descifren fácilmente el texto cifrado mediante ataques de fuerza bruta u otras técnicas.
3. **Mantener las bibliotecas y las implementaciones criptográficas actualizadas:** Actualice regularmente sus bibliotecas e implementaciones criptográficas para asegurar que estén libres de vulnerabilidades conocidas. Con el tiempo se pueden descubrir nuevas vulnerabilidades o debilidades, por lo que mantenerse actualizado es esencial para mantener un alto nivel de seguridad.

1.6 Ataques asociados a ChaCha20 y cifrados de flujo

Ataque	Descripción	Consecuencia	Prevención
Reutilización del Nonce (Nonce Reuse Attack)	Ocurre cuando se utiliza la misma clave secreta y el mismo nonce en diferentes mensajes, generando el mismo keystream.	Un atacante puede combinar los textos cifrados y obtener información de los mensajes originales.	Nunca reutilizar el nonce con la misma clave. Cada mensaje debe tener un nonce único.
Ataque de Texto Conocido (Known Plaintext Attack)	Si el atacante conoce parte del mensaje original y su versión cifrada, puede calcular parte del keystream utilizado.	Si el keystream se reutiliza, otros mensajes cifrados podrían ser descifrados parcialmente.	Utilizar nonces únicos y correcta gestión del contador.
Ataque por Mala Implementación	Vulnerabilidades provocadas por errores en la implementación del algoritmo, como generación incorrecta del nonce o ausencia de autenticación.	Permite manipulación o análisis del mensaje aunque el algoritmo sea seguro matemáticamente.	Usar implementaciones seguras como ChaCha20-Poly1305 y buenas prácticas criptográficas.

2. AES (Advanced Encryption Standard)

a. Introducción

El origen del algoritmo AES (Advanced Encryption Standard) responde a una necesidad crítica de seguridad informática a finales del siglo XX. Durante décadas, el estándar predominante fue el DES (Data Encryption Standard), el cual utilizaba claves de 56 bits. Sin embargo, con el avance de la capacidad de

cómputo, este algoritmo comenzó a mostrar vulnerabilidades graves ante ataques de fuerza bruta, como los conocidos "desafíos DES" lanzados por RSA, que demostraron que el estándar de la época ya no era capaz de proteger información sensible de manera efectiva.

Ante esta situación, en el año 1997, el Instituto Nacional de Estándares y Tecnología (NIST) de los Estados Unidos convocó a un concurso internacional con el objetivo de desarrollar un nuevo algoritmo de cifrado simétrico que sustituyera al DES. Los requisitos para el nuevo estándar eran rigurosos: debía ofrecer una seguridad significativamente mayor, ser eficiente tanto en implementación de hardware como de software y ser capaz de operar en dispositivos con recursos limitados, como tarjetas inteligentes.

Al concurso se presentaron 15 algoritmos candidatos provenientes de diversos países. Tras un exhaustivo proceso de evaluación técnica y criptoanálisis que duró más de dos años, en octubre del año 2000, el NIST proclamó como vencedor al algoritmo denominado Rijndael. Este algoritmo fue diseñado por los criptógrafos e investigadores belgas Joan Daemen y Vincent Rijmen. La elección se basó en que Rijndael ofrecía el mejor equilibrio entre seguridad, flexibilidad y rendimiento, convirtiéndose oficialmente en el estándar AES mediante la publicación del FIPS PUB 197 en el año 2001.

b. Características Técnicas y Funcionamiento del Algoritmo AES

El Advanced Encryption Standard (AES) se define técnicamente como un algoritmo de cifrado de bloque simétrico. A diferencia de su predecesor (DES), que utilizaba una red de Feistel, AES se basa en una red de sustitución-permutación, lo que le permite procesar el bloque de datos completo de manera más eficiente.

i. Especificaciones de Bloqueo y Clave

AES tiene una estructura flexible pero estandarizada en cuanto a sus dimensiones:

Tamaño de Bloque: El algoritmo opera siempre sobre bloques de datos de 128 bits (16 bytes). Estos datos se organizan internamente en una

matriz de estado de 4x4, donde cada celda representa un byte.

Longitud de Clave: El estándar admite tres longitudes de clave distintas, lo que determina el nivel de seguridad y la complejidad computacional:

- **AES-128:** Utiliza una clave de 128 bits y ejecuta 10 rondas de cifrado.
- **AES-192:** Utiliza una clave de 192 bits y ejecuta 12 rondas de cifrado.
- **AES-256:** Utiliza una clave de 256 bits y ejecuta 14 rondas de cifrado.

ii. **El Proceso de Cifrado (Rondas de Transformación)**

El proceso comienza con una etapa inicial llamada **AddRoundKey**, donde se realiza una operación XOR entre el bloque de texto en claro y la clave principal. Posteriormente, el algoritmo ejecuta una serie de rondas repetitivas (10, 12 o 14 según la clave) que constan de cuatro transformaciones fundamentales:

- **SubBytes (Sustitución):** Es una etapa de sustitución no lineal donde cada byte de la matriz de estado se reemplaza por otro utilizando una tabla de búsqueda predefinida llamada S-Box. Esta fase proporciona la propiedad de "confusión" en el cifrado.
- **ShiftRows (Desplazamiento de filas):** En esta etapa, las filas de la matriz de estado se desplazan cíclicamente hacia la izquierda. La primera fila permanece intacta, la segunda se desplaza un byte, la tercera dos bytes y la cuarta tres bytes. Esto garantiza que los bytes de cada columna se difundan por las demás.
- **MixColumns (Mezcla de columnas):** Es una operación de mezcla basada en aritmética de cuerpos de Galois. Cada columna se multiplica por un polinomio fijo, transformando los bytes de la columna para que el resultado dependa de todos los bytes de dicha columna. Nota importante: Esta operación se omite en la última ronda del algoritmo.
- **AddRoundKey (Adición de la clave de ronda):** En cada vuelta,

se genera una subclave distinta a partir de la clave maestra mediante un proceso llamado "Expansión de Clave". Esta subclave se aplica a la matriz de estado mediante una operación XOR bit a bit.

iii. Proceso de Descifrado

El descifrado de AES no es idéntico al cifrado, sino que consiste en ejecutar las operaciones inversas en orden contrario. Para recuperar el texto original, se aplican las funciones: InvShiftRows, InvSubBytes, AddRoundKey e InvMixColumns, utilizando las subclaves en orden inverso al que fueron generadas.

3. IDEA

3.1 Introducción

Las siglas IDEA significan International Data Encryption Algorithm (Algoritmo Internacional de Cifrado de Datos). Es un cifrador de bloques simétrico desarrollado en 1991 por los criptógrafos Xuejia Lai y James Massey. Este algoritmo opera en :

- **Bloque de datos:** 64 bits.
- **Longitud de clave:** 128 bits (conocida por su alta resistencia a los ataques criptograficos).
- **Estructura de Rondas:** Consta de 8 transformaciones idénticas (rondas completas) y una transformación de salida final conocida como "media ronda", sumando un total de 8.5 rondas.

Fuente: J. Guerrero-Reyes, C. Rodríguez-Martínez "IDEA: Algoritmo Criptográfico Simétrico para la Protección Segura de Datos Sensibles", no. 2, pág. 116, 2023. <https://revistas.udistrital.edu.co/index.php/vinculos/article/view/17635/21034>.

Evolución Histórica: El desarrollo del algoritmo comenzó en 1990 en la Escuela Politécnica Federal de Zúrich. Su primera versión fue presentada bajo el nombre de PES (Proposed Encryption Standard). En 1991, tras realizar mejoras para fortalecerlo contra el criptoanálisis diferencial, los autores presentaron el IPES (Improved Proposed Encryption Standard).

Finalmente, en 1992, el diseño fue rebautizado como IDEA. En 1999, tras exhaustivas pruebas de seguridad, se consolidó como un algoritmo mucho más robusto que el estándar DES. Su aplicación práctica más destacada y masiva es en el software de privacidad PGP (Pretty Good Privacy), donde se encarga del cifrado de alta velocidad de los datos.

Fuente: Ariel Santillan "Método de encriptación IDEA", pág. 2, 2013. <https://prezi.com/yz-cuizcqke/metodo-de-encriptacion-idea/>

3.2 El Modelo Matemático: El "Porqué" de las Fórmulas

Lo que hace especial a IDEA es que mezcla operaciones de tres grupos algebraicos diferentes. La seguridad reside en que estas operaciones no son "distributivas" ni "asociativas" entre sí, lo que genera una gran confusión y difusión de los datos.

Las 3 Operaciones Fundamentales:

- **XOR Bit a bit ($\oplus$):** Operación lógica estándar.
- **Suma módulo 2^{16} ($\boxplus$):** Suma de números de 16 bits ignorando el acarreo.
- **Multiplicación módulo $2^{16} + 1$ ($\odot$):** Esta es la más compleja. Se trata el bloque de 0000h como 2^{16} para asegurar que el módulo sea un número primo y así cada elemento tenga un inverso.

Fuente: Manuel J. Lucena López, "CRIPTOGRAFÍA Y SEGURIDAD EN COMPUTADORES", pág. 189, 2013. <https://ccia.esei.uvigo.es/docencia/SSI/cripto.pdf>

3.3 Funcionamiento: Cifrado y Descifrado

Generación de Subclaves (Key Schedule)

A partir de la clave de **128 bits**, se deben generar **52 subclaves** de 16 bits cada una.

- Las primeras 8 subclaves se toman directamente de los 128 bits.
- Luego, se realiza una rotación a la izquierda de 25 bits y se extraen las siguientes 8, y así sucesivamente.

Estructura de una Ronda

Cada bloque de 64 bits se divide en 4 sub-bloques de 16 bits (X_1, X_2, X_3, X_4). En cada ronda ocurre lo siguiente:

1. Multiplicaciones y sumas de los sub-bloques con las subclaves correspondientes.
2. Una estructura llamada MA (Multiplication-Addition) que mezcla los resultados.
3. Intercambio de los sub-bloques internos (excepto en la última ronda).

Descifrado

El proceso es idéntico al cifrado, pero las subclaves se usan en orden inverso y deben ser sus **inversos matemáticos** (inversos aditivos o multiplicativos según la operación).

Fuente: Manuel J. Lucena López, "CRIPTOGRAFÍA Y SEGURIDAD EN COMPUTADORES", pág. 199, 2013. <https://ccia.esei.uvigo.es/docencia/SSI/cripto.pdf>

3.4 Análisis de Seguridad y Ataques

Método de Ataque	Descripción
Fuerza Bruta	Con una clave de 128 bits (2^{128} combinaciones), es computacionalmente imposible de romper hoy en día.
Criptoanálisis Diferencial	IDEA es muy resistente. Se requieren ataques complejos sobre versiones reducidas (de pocas rondas) para ver debilidades.
Claves Débiles	Existen ciertas claves que pueden facilitar el análisis, aunque la probabilidad de generar una al azar es casi nula.

3.5 Ventajas y Desventajas

Ventajas:

- Velocidad: Muy eficiente en software y hardware.
- Sin S-Boxes: A diferencia de AES o DES, no usa tablas de sustitución, lo que ahorra memoria.

- Seguridad: No se conocen ataques prácticos que lo rompan en su versión completa de 8.5 rondas.

Desventajas:

- Tamaño de bloque: Al ser de 64 bits, es susceptible a "ataques de colisión" si se cifran volúmenes masivos de datos (terabytes) con la misma clave.
- Patentes: Estuvo bajo patente mucho tiempo, lo que limitó su adopción frente a AES.

Fuente: Lucena López, M. J. (2013). Criptografía y Seguridad en Computadores. Universidad de Jaén. pág. 199, <https://ccia.esei.uvigo.es/docencia/SSI/cripto.pdf>

4. Salsa20

1 Introducción

Salsa20 es un cifrador de flujo (*stream cipher*) de alto rendimiento diseñado por Daniel J. Bernstein en 2005. Fue presentado en el proyecto eSTREAM de la Unión Europea, siendo seleccionado para el portafolio final en la categoría de software (2008), lo cual valida su calidad criptográfica a nivel industrial.

El algoritmo genera un *keystream* pseudoaleatorio que se combina con el mensaje original mediante la operación XOR ($\oplus$), haciendo que los procesos de cifrado y descifrado sean simétricos. Su diseño se basa exclusivamente en operaciones aritméticas de tiempo constante (suma modular, rotación de bits y XOR), evitando el uso de S-boxes y mitigando así vulnerabilidades asociadas a ataques de canal lateral basados en temporización.

2. Características Técnicas

La arquitectura de Salsa20 opera sobre palabras de 32 bits, organizadas en una matriz de estado de 4×4 . Las especificaciones principales se resumen a continuación:

- Estado interno:** Matriz de 16 palabras de 32 bits (512 bits totales).
- Clave (K):** Soporta longitudes de 256 bits (recomendado) o 128 bits.
- Nonce (v):** Valor único de 64 bits por mensaje; su reutilización con la misma clave compromete la seguridad.
- Contador (t):** Valor de 64 bits que permite acceso aleatorio y procesamiento paralelo de

bloques.

- E. **Paradigma ARX:** Seguridad basada en **Addition** (suma modular), **Rotation** (rotación) y **XOR**.

3. Definición del Estado Inicial

La matriz de estado se inicializa entrelazando constantes, clave, nonce y contador en formato *little-endian*:

$$S = \begin{pmatrix} c_0 & k_0 & k_1 & k_2 \\ k_3 & c_1 & v_0 & v_1 \\ t_0 & t_1 & c_2 & k_4 \\ k_5 & k_6 & k_7 & c_3 \end{pmatrix}$$

Las constantes $\oplus$ derivan de la cadena ASCII "expand 32-byte k" y se ubican en la diagonal principal como valores *nothing-up-my-sleeve*, garantizando que el estado inicial nunca sea trivialmente débil.

4. Análisis del Algoritmo

4.1. Operaciones ARX

La resistencia criptográfica de Salsa20 emerge de la composición no lineal de tres operaciones fundamentales:

- **Suma modular** ($\boxplus$): Definida como $x \boxplus y := (x + y) \bmod 2^{32}$.
- **Rotación circular** ($\text{ROTL}_n(x)$): $\text{ROTL}_n(x) := (x \ll n) \vee (x \gg (32 - n))$.
Propaga cambios locales a través de toda la palabra (difusión).
- **XOR** ($\oplus$): Operación lineal bit a bit que aporta confusión algebraica.

4.2. Función Quarter Round (QR)

La unidad básica de mezcla es la *Quarter Round* (QR), que transforma un vector de cuatro palabras (a, b, c, d) mediante la siguiente secuencia de operaciones:

$$b \leftarrow b \oplus \text{ROTL}_7(a \boxplus d)$$

$$c \leftarrow c \oplus \text{ROTL}_9(b \boxplus a)$$

$$d \leftarrow d \oplus \text{ROTL}_{13}(c \boxplus b)$$

$$a \leftarrow a \oplus \text{ROTL}_{18}(d \boxplus c)$$

Las constantes de rotación {7, 9, 13, 18} fueron seleccionadas para optimizar la difusión criptográfica. La dependencia secuencial entre operaciones acelera el efecto avalancha, requisito esencial para la resistencia frente a criptoanálisis diferencial y lineal.

4.3. Estructura de Rondas

El estado completo se procesa mediante la aplicación alternada de:

- **Column Round:** Aplicación de la función *Quarter Round* (QR) a cada columna de la matriz de estado.
- **Row Round:** Aplicación de QR a cada fila de la matriz resultante.

Una *doble ronda* consiste en una Column Round seguida de una Row Round. El estándar Salsa20 ejecuta 10 dobles rondas (20 iteraciones simples), equilibrio validado entre seguridad y rendimiento.

4.4. Feedforward Final Blindaje Opcional

Tras completar las rondas, el estado transformado W se combina con el estado inicial S mediante suma modular elemento a elemento:

$$\text{Salsa20}(S) = (S_0 \boxplus W_0, \dots, S_{15} \boxplus W_{15})$$

Este mecanismo de *feedforward* garantiza que la función sea computacionalmente no invertible, impidiendo la recuperación de la clave a partir de salidas observadas.

5 Cifrado y Descifrado

Salsa20 es un cifrador de flujo (stream cipher), lo que significa que no opera sobre bloques aislados, sino que genera una secuencia de bits pseudoaleatoria denominada keystream a partir de la clave, el nonce y un contador. Tanto el cifrado como el descifrado se realizan con la misma operación: un XOR bit a bit entre el keystream y los datos.

Proceso de Cifrado

1. El mensaje se divide en bloques de 64 bytes (512 bits), igual al tamaño de salida de la función Salsa20.
2. Para cada bloque i , se construye el estado inicial usando la clave K , el nonce v y el contador $t=i$.
3. Se ejecutan las 20 rondas y se aplica el feedforward para obtener un bloque de keystream Z_i
4. Se calcula el texto cifrado: $C_i = M_i \oplus Z_i$.
5. Si el último bloque es menor a 64 bytes, se trunca el keystream correspondiente antes de aplicar el XOR.
6. Se incrementa el contador y se repite el proceso hasta cubrir todo el mensaje.

Proceso de Descifrado

1. Con la misma clave K , nonce v y contador $t=i$, se regenera exactamente el mismo keystream Z_i
2. Se aplica: $M_i = C_i \oplus Z_i$
3. Se repite bloque a bloque hasta recuperar el mensaje original

6. Mini-Salsa20

Con el propósito de facilitar la comprensión de los principios ARX sin la complejidad computacional de palabras de 32 bits, este informe incorpora una abstracción didáctica denominada **Mini-Salsa20**. Esta variante conceptual reduce el dominio de operación a nibbles (4 bits) y módulo 16, preservando la estructura algebraica del algoritmo original.

Características de la abstracción:

- A. Operaciones aritméticas y lógicas análogas, adaptadas al dominio reducido.
- B. Rotaciones circulares sobre espacios de 4 bits para ilustrar el mecanismo de difusión.
- C. Priorización de la claridad conceptual sobre la ejecución manual.

Esta aproximación permite identificar cómo la composición de suma modular, XOR y rotación genera no linealidad y difusión, principios que escalan directamente a la implementación real.

7. Consideraciones de Seguridad

- **Gestión de Nonce:** La reutilización de un par (Clave, Nonce) permite a un atacante obtener $M_1 \oplus M_2$, comprometiendo la confidencialidad.
- **Variantes de Rondas Reducidas:** Salsa20/8 y Salsa20/12 ofrecen menor costo computacional a expensas de margen de seguridad. Solo la variante de 20 rondas está recomendada para aplicaciones críticas.
- **Integridad de Datos:** Salsa20 provee confidencialidad pero no autenticación. Su uso debe complementarse con un mecanismo de autenticación (Poly1305) en modo AEAD.

5. TwoFish

5.1. Origen

Twofish fue presentado en 1998 como una evolución de Blowfish. Su objetivo era cumplir con los requisitos del NIST para el nuevo estándar de cifrado: un bloque de 128 bits y llaves de hasta 256 bits. Aunque finalmente el NIST eligió a Rijndael (AES), Twofish sigue siendo considerado uno de los algoritmos más seguros y robustos, destacando por sus cajas S dependientes de la clave.

Referencia: Schneier, B., et al. (1998). *Twofish: A 128-Bit Block Cipher*. Counterpane Systems.

5.2. Arquitectura Matemática: La Red de Feistel

Twofish utiliza una Red de Feistel de 16 rondas, similar a DES, pero con una vuelta de tuerca: aplica una permutación adicional llamada "blanqueo" (whitening) al principio y al final. El bloque de 128 bits se divide en cuatro palabras de 32 bits (w_0, w_1, w_2, w_3). En cada ronda, se utilizan dos palabras para modificar las otras dos mediante una función F :

$$R_{i,0}, R_{i,1} = F(R_{i-1,0}, R_{i-1,1}, \text{ronda}, K)$$

Donde la función F es una estructura compleja que combina tres conceptos matemáticos clave: Cajas S, Matrices MDS y el PHT.

Referencia: NIST. (2000). *Report on the Development of the Advanced Encryption Standard (AES)*. National Institute of Standards and Technology.

5.3. Componentes de la Función F y su Justificación

A. Cajas S Dependientes de la Clave

A diferencia de DES (donde las cajas S son fijas), en Twofish las cajas S se generan matemáticamente a partir de la llave del usuario.

- ¿Por qué? Esto hace que el algoritmo sea inmune a ataques de criptoanálisis diferencial genérico, ya que el atacante no conoce las tablas de sustitución de antemano. Se requiere un análisis específico para cada llave.

B. Matriz MDS (Maxima Distancia Separable)

Se utiliza una multiplicación de matrices sobre un Campo de Galois $GF(2^8)$. La matriz MDS tiene una propiedad matemática vital: cualquier cambio en un bit de entrada garantiza un cambio máximo en los bits de salida.

- ¿Por qué? Proporciona una difusión perfecta. Matemáticamente, si la matriz es M y el vector de entrada es V , la salida es $Y = M * V$. La distancia de Hamming se maximiza aquí para evitar ataques lineales.

Referencia: Daemen, J., & Rijmen, V. (2002). *The Design of Rijndael: AES - The Advanced Encryption Standard*. Springer.

C. PHT (Pseudo-Hadamard Transform)

Es una operación aritmética simple y rápida sobre palabras de 32 bits:

$$a' = (a + b) \pmod{2^{32}}$$

$$b' = (a + 2b) \pmod{2^{32}}$$

5.4. El Proceso de Descifrado

Al ser una red de Feistel, el descifrado es idéntico al cifrado, pero aplicando las subclaves en orden inverso. La propiedad matemática del XOR permite que la estructura se mantenga:

$$X \oplus Y \oplus Y = X$$

Esto simplifica la implementación en sistemas embebidos donde el espacio de memoria es limitado.

Referencia: Menezes, A. J., et al. (1996). *Handbook of Applied Cryptography*. CRC Press.

5.5. Aplicaciones de Twofish en la Actualidad (2026)

A pesar de que el NIST estandarizó el AES, Twofish no ha muerto; de hecho, es una de las alternativas más respetadas cuando se busca evitar el "monocultivo criptográfico" (el riesgo de que todos usen lo mismo y se descubra una vulnerabilidad global).

A. Cifrado de Discos y Particiones (VeraCrypt)

El uso más famoso de Twofish hoy en día es en VeraCrypt (el sucesor espiritual de TrueCrypt).

- **Cifrado en Cascada:** VeraCrypt permite usar Twofish junto con otros algoritmos. Muchos usuarios avanzados eligen configuraciones como AES + Twofish + Serpent.
- **Justificación:** Si uno de los algoritmos fuera vulnerado mediante un avance matemático secreto, los otros dos seguirían protegiendo los datos.

B. Privacidad en Correos Electrónicos (GnuPG / PGP)

El estándar OpenPGP (usado por herramientas como GnuPG) incluye a Twofish entre sus algoritmos soportados. Se utiliza para el cifrado simétrico de los paquetes de datos

de correo electrónico. Es común verlo en entornos donde se requiere una comunicación ultra segura entre servidores de diferentes arquitecturas.

C. Seguridad en Redes y VPNs (OpenVPN)

Aunque AES y ChaCha20 dominan el tráfico web (TLS), protocolos como OpenVPN permiten configurar Twofish para el túnel de datos. Es utilizado especialmente en infraestructuras críticas donde se prefiere un algoritmo que no haya sido "influenciado" por cambios de último minuto de agencias gubernamentales (recordando la historia de las cajas S del DES).

Referencia: VeraCrypt Team. (2026). *VeraCrypt Documentation: Cascaded Encryption Algorithms*. / Callas, J., et al. (2007). *RFC 4880: OpenPGP Message Format*.

5.6. Ventajas y Desventajas

Ventajas

- **Seguridad Extrema:**

La seguridad de Twofish no es solo una cuestión de "fuerza bruta" por el tamaño de su llave (que llega a 256 bits, ofreciendo un espacio de búsqueda de 2^{256} combinaciones).

Su verdadera fortaleza reside en que las S-Boxes son dependientes de la clave

- **Flexibilidad:**

Twofish es único porque permite al ingeniero decidir cómo quiere que se comporte el algoritmo según los recursos del sistema. Esto se conoce como Compromiso Espacio-Tiempo (Space-Time Trade-off).

Referencia: Lipmaa, H. (1999). *On Differential Properties of the Twofish PHT and MDS*. University of Tartu.

Desventajas

- **Rondas Lentas:**

Cada una de las 16 rondas de Twofish es "pesada" comparada con otros

algoritmos. Mientras que AES realiza operaciones muy simples que se pueden ejecutar en paralelo, la función F de Twofish tiene varios pasos secuenciales que no se pueden saltar

- **Complejidad del Key Schedule:**

El *Key Schedule* es el proceso de preparar las 40 subclaves que Twofish necesita para sus rondas. En Twofish, este proceso es deliberadamente lento.

Referencia: Lipmaa, H. (1999). *On Differential Properties of the Twofish PHT and MDS*. University of Tartu.

CRIPTOGRAFÍA ASIMÉTRICA

1. RSA

1.1 Introducción

RSA (Rivest-Shamir-Adleman) es un algoritmo de criptografía asimétrica nombrado en honor a sus tres creadores. Fue presentado públicamente en agosto de 1977 en la columna 'Mathematical Games' de la revista Scientific American, y una patente fue otorgada al MIT el 20 de septiembre de 1983 (U.S. Patent 4,405,829).

RSA fue el primer algoritmo conocido que podía utilizarse tanto para cifrar mensajes como para crear firmas digitales. Su seguridad se basa en la dificultad computacional de factorizar el producto de dos números primos muy grandes, un problema matemático para el cual no se conoce ningún algoritmo eficiente en computadoras clásicas.

Desde su publicación, RSA se ha convertido en uno de los estándares de facto de la seguridad informática y es utilizado en numerosos protocolos y aplicaciones, incluyendo SSL/TLS (HTTPS), SSH, PGP, S/MIME, y muchos otros sistemas de seguridad.

1.2 Fundamentos Matemáticos

La seguridad de RSA descansa sobre dos pilares matemáticos fundamentales: la aritmética modular y la dificultad de la factorización de números enteros grandes.

1.2.1 Aritmética Modular

La operación básica es la exponenciación modular: dado un mensaje M , se calcula $M^e \bmod n$ para cifrar, o $C^d \bmod n$ para descifrar. El operador 'mod' devuelve el residuo de una división entera. Por ejemplo, $7 \bmod 3 = 1$, ya que $7 = 2 \times 3 + 1$.

1.2.2 La Función de Euler (Totiente)

La función $\Phi(n)$ (phi de n), llamada función totiente de Euler, cuenta cuántos enteros positivos menores o iguales a n son coprimos con n . Para el caso especial donde $n = p \times q$ (producto de dos primos distintos), se cumple que: $\Phi(n) = (p-1) \times (q-1)$. Esta propiedad es esencial en la generación de claves RSA.

1.2.3 El Problema de Factorización

Si se conoce n (que es público), teóricamente se podría encontrar p y q factorizando n . Sin embargo, cuando p y q son números primos con cientos o miles de dígitos, la factorización de su producto se vuelve computacionalmente inviable con la tecnología actual. Por ejemplo, factorizar un número de 2048 bits requeriría más tiempo del que lleva existiendo el universo con las computadoras actuales.

1.3. Generación de Claves

La generación del par de claves pública y privada es el proceso más complejo del algoritmo RSA. Los pasos son los siguientes:

Paso 1: Seleccionar dos números primos grandes p y q . Estos valores deben ser elegidos al azar y mantenerse en secreto. En implementaciones reales, se utilizan números con cientos de dígitos.

Paso 2: Calcular $n = p \times q$. Este valor n es el módulo que forma parte tanto de la clave pública como de la privada.

Paso 3: Calcular la función totiente: $\Phi(n) = (p-1) \times (q-1)$. Este valor también se mantiene secreto.

Paso 4: Elegir el exponente público e , tal que: $1 < e < \Phi(n)$ y $\text{mcd}(e, \Phi(n)) = 1$ (es decir, e y $\Phi(n)$ son coprimos). Un valor muy común en la práctica es $e = 65537$, que es primo y no

demasiado grande.

Paso 5: Calcular el exponente privado d , tal que: $(d \times e) \equiv 1 \pmod{\Phi(n)}$. Es decir, d es el inverso multiplicativo modular de e respecto a $\Phi(n)$. Esto se calcula usando el Algoritmo de Euclides Extendido.

Al finalizar el proceso:

- Clave Pública = (n, e) — se comparte libremente con todos.
- Clave Privada = (n, d) — se mantiene en secreto absoluto.
- Los valores p , q y $\Phi(n)$ se descartan o se guardan con máxima protección.

1.4. Proceso de Cifrado y Descifrado

1.4.1 Cifrado

Para cifrar un mensaje M (representado como un número entero donde $0 \leq M < n$), el emisor usa la clave pública del receptor (n, e) y aplica la fórmula:

$$C = M^e \pmod{n}$$

Donde C es el texto cifrado resultante. Cualquier persona que tenga la clave pública puede cifrar mensajes destinados al receptor.

1.4.2 Descifrado

Para descifrar el texto cifrado C , el receptor usa su clave privada (n, d) y aplica la fórmula:

$$M = C^d \pmod{n}$$

Solo el receptor, que posee la clave privada d , puede recuperar el mensaje original M . Incluso quien cifró el mensaje no puede descifrarlo sin la clave privada.

1.5 Ejemplo Numérico Simplificado

A continuación se muestra un ejemplo con números pequeños para ilustrar el proceso. En la práctica real se usan números mucho más grandes:

Generación de Claves:

- Seleccionamos $p = 11$ y $q = 13$ (primos pequeños para el ejemplo).
- Calculamos $n = 11 \times 13 = 143$.
- Calculamos $\Phi(n) = (11-1) \times (13-1) = 10 \times 12 = 120$.

- Elegimos $e = 7$ (coprimo con 120, ya que $\text{mcd}(7, 120) = 1$).
- Calculamos d tal que $(d \times 7) \equiv 1 \pmod{120}$. El resultado es $d = 103$, ya que $7 \times 103 = 721 = 6 \times 120 + 1$.
- Clave pública: (143, 7). Clave privada: (143, 103).

Cifrado del mensaje $M = 9$:

- $C = 9^7 \pmod{143}$
- $9^7 = 4,782,969$
- $4,782,969 \pmod{143} = 48$
- Texto cifrado: $C = 48$

Descifrado del texto cifrado $C = 48$:

- $M = 48^{103} \pmod{143}$
- Aplicando exponenciación modular eficiente (algoritmo de cuadrados y multiplicaciones):
- $M = 9 \leftarrow$ se recupera el mensaje original.

1.6. Aplicaciones Prácticas del RSA

RSA es ampliamente utilizado en sistemas de seguridad actuales, tanto de forma directa como en combinación con otros algoritmos:

1.6.1 Protocolo SSL/TLS (HTTPS)

Cuando un navegador se conecta a un sitio web seguro (HTTPS), se realiza un proceso conocido como TLS Handshake. En este proceso, RSA se utiliza para autenticar la identidad del servidor y para intercambiar de forma segura una clave simétrica temporal que se usará para cifrar el resto de la comunicación. Este enfoque híbrido combina la seguridad del intercambio de claves RSA con la eficiencia del cifrado simétrico (como AES).

1.6.2 SSH (Secure Shell)

SSH utiliza RSA para la autenticación de usuarios y servidores. Las claves SSH RSA permiten el acceso remoto seguro a sistemas sin necesidad de contraseñas, utilizando un par de claves: la pública almacenada en el servidor y la privada guardada en el cliente.

1.6.3 Correo Electrónico Seguro (PGP/GPG y S/MIME)

PGP (Pretty Good Privacy) y su versión de código abierto GPG (GNU Privacy Guard) utilizan

RSA para el cifrado y la firma digital de correos electrónicos. S/MIME (Secure/Multipurpose Internet Mail Extensions) también emplea RSA en entornos corporativos para proteger las comunicaciones por email.

1.6.4 Firma Digital de Código y Software

Los fabricantes de software usan RSA para firmar digitalmente sus aplicaciones y actualizaciones. Esto garantiza que el código no ha sido modificado maliciosamente en tránsito y que proviene de una fuente confiable. El sistema operativo verifica esta firma antes de instalar software.

1.6.5 Banca Electrónica y Comercio Electrónico

Las plataformas de banca en línea y comercio electrónico dependen de RSA para proteger las transacciones financieras. Cada vez que se realiza un pago en línea, RSA está trabajando en segundo plano para garantizar que la información de la tarjeta de crédito y los datos personales estén protegidos.

1.7 Tres Ataques Específicos al Algoritmo RSA

1.7.1 Ataque de Factorización por Fuerza Bruta

Este es el ataque más directo contra RSA. Si un atacante puede factorizar el módulo público n en sus factores primos p y q , puede calcular $\Phi(n) = (p-1)(q-1)$, y con ello encontrar la clave privada d a partir del exponente público e .

¿Cómo funciona? El atacante intenta dividir n por todos los números primos posibles hasta encontrar p y q . El algoritmo más eficiente conocido actualmente para este fin es el Criba del Campo Numérico General (Number Field Sieve, NFS). Sin embargo, incluso con este algoritmo, factorizar una clave de 2048 bits requeriría una cantidad astronómica de tiempo y recursos computacionales.

Resultados históricos: Investigadores lograron factorizar una clave RSA de 768 bits en 2010, pero el proceso tomó 2 años, miles de horas de cómputo y una cantidad enorme de recursos. Las claves de 1024 bits ya no se consideran seguras y las de 2048 bits son el estándar mínimo actual.

Constramedida: Utilizar claves de al menos 2048 bits, aunque el estándar recomendado actualmente es 4096 bits para mayor seguridad a largo plazo.

1.7.2 Ataque de Temporización (Timing Attack)

Este es un ataque de canal lateral (side-channel attack) descubierto por Paul Kocher en 1996. En lugar de atacar el algoritmo matemático directamente, explota variaciones en el tiempo de ejecución de las operaciones criptográficas.

¿Cómo funciona? La operación principal en RSA es la exponenciación modular, que se implementa típicamente mediante el algoritmo de 'cuadrados y multiplicaciones'. Esta implementación tarda tiempos ligeramente distintos dependiendo de los bits de la clave privada d : cuando el bit es 1, se realiza una multiplicación adicional. Midiendo con precisión estos tiempos para miles de operaciones, un atacante puede deducir bit a bit el valor de la clave privada.

Caso real: En 2003, Boneh y Brumley demostraron un ataque de temporización contra servidores web OpenSSL a través de la red local, logrando extraer la clave privada RSA de 1024 bits en pocas horas con aproximadamente un millón de consultas.

Contramedida: Implementar 'blinding' (cegamiento), que introduce números aleatorios en las operaciones para hacer que el tiempo sea independiente de los datos. OpenSSL activó esta protección por defecto tras la demostración de Boneh y Brumley.

1.7.3 Ataque de Exponente Público Pequeño (Ataque de Håstad)

Este ataque, publicado por Johan Håstad, se produce cuando se usa un exponente público muy pequeño (típicamente $e = 3$) y el mismo mensaje M es enviado cifrado a múltiples destinatarios sin relleno aleatorio (padding).

¿Cómo funciona? Supongamos que el mensaje M se cifra para 3 destinatarios con módulos diferentes n_1, n_2, n_3 , usando $e = 3$. El atacante obtiene: $C_1 = M^3 \bmod n_1$, $C_2 = M^3 \bmod n_2$, $C_3 = M^3 \bmod n_3$. Aplicando el Teorema Chino del Resto, el atacante puede combinar estos tres textos cifrados para calcular M^3 directamente (sin módulo), y como $M < \min(n_1, n_2, n_3)^{1/3}$, la raíz cúbica de M^3 es simplemente M .

Contramedida: Utilizar esquemas de relleno aleatorio como OAEP (Optimal Asymmetric Encryption Padding), que introduce aleatoriedad antes del cifrado. OAEP evita que el mismo mensaje produzca el mismo texto cifrado en diferentes ocasiones, neutralizando este tipo de ataque.

1.8 Análisis de Seguridad y Comparación

RSA es considerado un algoritmo muy seguro cuando se implementa correctamente, pero presenta algunas limitaciones:

Fortalezas:

- Seguridad basada en un problema matemático (factorización) para el que no existe solución eficiente conocida en computación clásica.
- Permite tanto cifrado como firma digital con el mismo par de claves.
- Ampliamente estandarizado y soportado en todas las plataformas y bibliotecas criptográficas.
- Permite el intercambio seguro de claves sin necesidad de un canal seguro previo.

Debilidades:

- Rendimiento: RSA es significativamente más lento que los algoritmos simétricos como AES, especialmente para datos grandes. Por eso se usa en modo híbrido.
- Vulnerabilidad a la computación cuántica: El algoritmo de Shor puede factorizar números grandes de forma eficiente en una computadora cuántica, lo que haría RSA obsoleto.
- Dependencia del tamaño de clave: A medida que aumenta el poder computacional, se necesitan claves más largas, lo que incrementa el tiempo de procesamiento.
- Vulnerabilidades de implementación: RSA implementado sin padding adecuado (como en el ataque de Håstad) o sin protecciones de canal lateral es inseguro.

Comparación con otros algoritmos asimétricos:

- ECC (Elliptic Curve Cryptography): Ofrece seguridad equivalente con claves mucho más pequeñas (una clave ECC de 256 bits equivale a una RSA de 3072 bits), siendo más eficiente.
- ElGamal: Basado en el problema del logaritmo discreto, ofrece propiedades homomórficas pero produce textos cifrados el doble de grandes.
- Diffie-Hellman: No cifra directamente, sino que permite el intercambio de claves; más eficiente que RSA para este propósito específico.

2. Criptosistema de Paillier

1. Introducción

El Criptosistema de Paillier es un algoritmo asimétrico probabilístico de clave pública, inventado y presentado por Pascal Paillier en 1999 en el artículo titulado "Public-Key Cryptosystems Based on Composite Degree Residuosity Classes", presentado en la conferencia EUROCRYPT '99 y publicado por Springer.

Este sistema se ubica dentro de la categoría de la criptografía asimétrica moderna y se distingue por su propiedad de cifrado homomórfico aditivo: dadas únicamente la clave pública y los textos cifrados de dos mensajes, es posible calcular el cifrado de su suma sin necesidad de descifrar los mensajes originales. Esta característica lo convierte en una herramienta fundamental en aplicaciones de privacidad computacional.

Paillier sigue siendo relevante y vigente en aplicaciones modernas de privacidad, votaciones electrónicas, computación segura en la nube, y sistemas de aprendizaje federado preservando la privacidad.

2. Fundamentos Matemáticos

El Criptosistema de Paillier se fundamenta en el problema de la Residuosity

Compuesta (Composite Residuosity Problem). Este problema consiste en determinar si un número dado pertenece a la clase de los n -ésimos residuos módulo n^2 , donde n es un número compuesto (producto de dos primos grandes).

2.1 Supuesto de Seguridad: DCRA

La seguridad computacional del sistema descansa sobre la Decisional Composite Residuosity Assumption (DCRA): se asume que ningún algoritmo eficiente puede distinguir si un elemento aleatorio de $Z^*_{\{n^2\}}$ es un n -ésimo residuo o no, sin conocer la factorización de n . Esta asunción se considera computacionalmente intratable según el estado actual del arte.

2.2 Función auxiliar $L(x)$

Una función clave en el algoritmo es:

$$L(x) = (x - 1) / n$$

Donde $x \equiv 1 \pmod{n}$ para que la división sea entera. Esta función permite extraer el mensaje durante el descifrado.

2.3 Función de Carmichael $\lambda(n)$

La función de Carmichael (también llamada lambda o indicatriz reducida) se calcula como:

$$\lambda(n) =$$

$$\text{mcm}(p-1, q-1)$$

mcm = mínimo común múltiplo. Esta función es la clave privada principal y es computacionalmente intratable obtener a partir de n sin conocer p y q .

3. Generación de Claves

La generación de claves en Paillier sigue los siguientes pasos, basados en la especificación original de Pascal Paillier (1999):

Paso 1: Selección de números primos

Se eligen dos números primos grandes p y q de forma aleatoria e independiente. La especificación recomienda que ambos sean de igual longitud en bits para garantizar que

$\gcd(pq, (p-1)(q-1)) = 1$, condición necesaria para la correcta operación del sistema. Se recomienda usar el algoritmo de Miller-Rabin para verificar la primalidad con alta confianza.

Recomendación NIST (2024)	Módulo mínimo de 2048 bits para seguridad de 112 bits, válida hasta 2030.
----------------------------------	---

Paso 2: Cálculo del módulo

$n = p \times q$	n es el módulo público. Su tamaño determina el nivel de seguridad.
------------------------------------	--

Paso 3: Cálculo de λ (clave privada)

$\lambda = \text{mcm}(p-1, q-1)$	Función de Carmichael de n . Es el componente principal de la clave privada.
--	--

Paso 4: Selección de g (generador)

Se elige un entero aleatorio g en el conjunto $Z^*_{\{n^2\}}$ (enteros entre 1 y $n^2 - 1$). Para la variante simplificada recomendada para implementación, se puede usar directamente $g = n + 1$, ya que simplifica los cálculos sin reducir la seguridad.

$g \in Z^*_{\{n^2\}}$	g es parte de la clave pública.
---	-----------------------------------

Paso 5: Cálculo de μ (clave privada auxiliar)

$\mu = (L(g^\lambda \bmod n^2))^{-1} \bmod n$	μ es el inverso modular multiplicativo. Se usa en el
---	--

	descifrado.
--	-------------

4. Proceso de Cifrado

Dado un mensaje m que se desea cifrar, donde $0 \leq m < n$, y usando la clave pública (n, g) :

Paso 1: Selección del valor aleatorio r

Se elige un entero aleatorio r tal que $1 \leq r < n$ y $\text{mcd}(r, n) = 1$. Este valor es el componente probabilístico del sistema: el mismo mensaje m cifrará diferente cada vez, lo que garantiza la seguridad semántica.

$r \in \mathbb{Z}_n^*$	r es el factor de aleatoriedad (nonce). Nunca se debe reutilizar r .
------------------------	--

Paso 2: Cálculo del texto cifrado

$c = g^m \cdot r^n \bmod n^2$	c es el texto cifrado. La operación se realiza módulo n^2 .
-------------------------------	---

El tamaño del texto cifrado c es aproximadamente el doble del tamaño de n (ciphertext expansion), lo que representa un costo en ancho de banda.

5. Proceso de Descifrado

Dado el texto cifrado c y la clave privada (λ, μ) :

Paso 1: Elevar c a la potencia λ módulo n^2

$u = c^\lambda \bmod n^2$	u es un valor intermedio usado para extraer el mensaje.
---------------------------	---

Paso 2: Aplicar la función L

$L(u) = (u - 1) / n$	División entera exacta, ya que $u \equiv 1 \pmod{n}$ por construcción.
----------------------	--

Paso 3: Recuperar el mensaje

$m = L(c^\lambda \bmod n^2) \cdot \mu \bmod n$	μ actúa como factor de normalización para obtener m.
--	--

La corrección del descifrado (que siempre se obtiene el mensaje original) está demostrada formalmente en el paper original de Paillier (1999), usando la expansión binomial de $(n+1)^k$.

6. Propiedad Homomórfica Aditiva

La propiedad más notable del Criptosistema de Paillier es su homomorfismo aditivo. Esta propiedad permite realizar operaciones aritméticas sobre datos cifrados sin necesidad de descifrarlos, lo que es fundamental para aplicaciones de privacidad.

6.1 Suma de mensajes cifrados

$E(m_1) \cdot E(m_2) \bmod n^2 = E(m_1 + m_2 \bmod n)$	El producto de dos cifrados es el cifrado de la suma de sus mensajes.
--	---

En términos concretos: si multiplicamos los dos textos cifrados (operación en $Z^*_{\{n^2\}}$), al descifrar el resultado obtenemos la suma de los mensajes originales. Nadie que no tenga la clave privada puede conocer m_1 ni m_2 por separado.

6.2 Multiplicación de un cifrado por una constante

$E(m)^k \bmod n^2 = E(k \cdot m \bmod n)$	Elevar un cifrado a la potencia k produce el cifrado de k·m.
---	--

Esta propiedad es especialmente útil en protocolos de votación electrónica: si cada voto se codifica como 0 o 1, el producto de todos los votos cifrados es el cifrado del total de votos afirmativos.

6.3 Clasificación del Homomorfismo

El Criptosistema de Paillier es un sistema de cifrado parcialmente homomórfico (Partially Homomorphic Encryption - PHE): soporta operaciones de adición sobre textos cifrados, pero no multiplicación entre dos textos cifrados (lo cual requeriría un esquema de Cifrado Completamente Homomórfico - FHE, como BFV o CKKS).

EJEMPLO

FASE 0: GENERACIÓN DE CLAVES

Esta fase establece la infraestructura matemática necesaria.

- Selección de números primos: Se eligen $p = 17$ y $q = 19$.
- Verificación de condición: Se comprueba que el Máximo Común Divisor $MCD(323, 288) = 1$.
- Cálculo del módulo (n): $n = p \times q = 323$.
- Módulo de operación (n^2): $n^2 = 323 \times 323 = 104.329$.
- Cálculo de λ (Clave Privada): Es el Mínimo Común Múltiplo de $(p-1)$ y $(q-1)$. $MCM(16, 18) = 144$.
- Selección del generador (g): Se usa la variante simplificada $g = n + 1 = 324$.
- Cálculo de μ (Clave Privada auxiliar):
 - g elevado a $\lambda \bmod n^2 = 324^{144} \bmod 104.329 = 46.513$.
 - Función $L(46.513) = (46.513 - 1) / 323 = 144$.
 - $\mu = \text{inverso modular de } 144 \bmod 323 = 83$.
 - Verificación: $144 \times 83 \bmod 323 = 1$.

Resumen de Claves:

- Clave Pública: ($n = 323$, $g = 324$)
- Clave Privada: ($\lambda = 144$, $\mu = 83$)

FASE 1: PROCESO DE CIFRADO ($m = 42$)

Se transforma el mensaje $m = 42$ en un texto cifrado utilizando la clave pública.

- Selección del valor aleatorio (r): Se elige $r = 13$. Se verifica que $\text{MCD}(13, 323) = 1$.
- Cálculo de factores:
 - g elevado a $m \bmod n^2 = 324^{42} \bmod 104.329 = 13.567$.
 - r elevado a $n \bmod n^2 = 13^{323} \bmod 104.329 = 75.756$.
- Texto Cifrado (c): $c = 13.567 \times 75.756 \bmod 104.329 = 36.673$.

FASE 2: PROCESO DE DESCIFRADO

Recuperación del mensaje original a partir del texto cifrado $c = 36.673$ usando la clave privada.

1. Cálculo de u : $u = c$ elevado a $\lambda \bmod n^2 = 36.673^{144} \bmod 104.329 = 75.583$.
2. Verificación interna: Se confirma que $u \bmod n = 75.583 \bmod 323 = 1$.
3. Aplicación de Función L : $L(75.583) = (75.583 - 1) / 323 = 234$.
4. Recuperación de m : $m = 234 \times 83 \bmod 323 = 42$.

FASE 3: VERIFICACIÓN HOMOMÓRFICA

Demostración de que el producto de dos cifrados equivale al cifrado de la suma.

1. Datos: Mensaje 1 ($m_1 = 42$, $c_1 = 36.673$). Mensaje 2 ($m_2 = 15$, $c_2 = 85.853$).
2. Operación homomórfica: Multiplicación de los textos cifrados módulo n^2 .
 - $c_{\text{suma}} = 36.673 \times 85.853 \bmod 104.329 = 46.507$.
3. Descifrado de verificación: Al descifrar 46.507 siguiendo los pasos de la Fase 2, el resultado es 57.

- o Conclusión: $m_1 + m_2 = 42 + 15 = 57$. La propiedad se cumple exitosamente.

7. Análisis de Seguridad

7.1 Seguridad demostrable

El criptosistema de Paillier es semánticamente seguro frente a ataques de texto plano elegido (IND-CPA: Indistinguishability under Chosen-Plaintext Attack). La capacidad de distinguir el texto cifrado del desafío equivale exactamente a resolver la DCRA, que se considera computacionalmente intratable.

La seguridad probabilística del sistema es una característica fundamental: dado que el valor r es aleatorio, el mismo mensaje m produce diferentes cifrados en cada ejecución, lo que evita ataques de diccionario.

7.2 Tres ataques específicos al Criptosistema de Paillier

Ataque 1: Ataque de Texto Cifrado Elegido Adaptativo (IND-CCA2)

El esquema básico de Paillier NO es seguro frente a ataques IND-CCA2. Debido a su propiedad homomórfica (maleabilidad), un adversario con acceso a un oráculo de descifrado puede modificar un texto cifrado c en uno relacionado $c' = c \cdot E(k) = E(m + k)$, y enviarlo al oráculo para obtener $m+k$, lo que le permite deducir m .

Mitigación: Aplicar padding OAEP (Optimal Asymmetric Encryption Padding) adaptado para Paillier, o utilizar la variante de Paillier-Pointcheval (ASIACRYPT'99) que incorpora hash combinado del mensaje m con el factor r , logrando seguridad IND-CCA2 en el modelo de oráculo aleatorio.

Ataque 2: Ataque por Factorización del Módulo n

Si un adversario logra factorizar $n = pq$ (obteniendo p y q), puede calcular directamente $\lambda = \text{lcm}(p-1, q-1)$ y μ , recuperando la clave privada completa. El algoritmo de criba del campo numérico (Number Field Sieve, NFS) es el método más eficiente conocido para factorización, con complejidad subexponencial. Según ScienceDirect Topics (Paillier Cryptosystem), esto implica que módulos de 1024 bits ya no son seguros, y se requieren al menos 2048 bits.

Mitigación: Usar módulos de al menos 2048 bits (recomendado por NIST para

seguridad hasta 2030) o 3072 bits para mayor margen. Generar p y q usando primos seguros (safe primes: $p = 2p' + 1$, donde p' también es primo) para aumentar la resistencia a variantes de factorización.

Ataque 3: Ataque de Maleabilidad en Protocolos

La maleabilidad del sistema Paillier — aunque intencional en algunas aplicaciones — puede ser explotada como ataque en contextos donde no se desea. Por ejemplo, en una subasta electrónica sellada, un adversario que observa la oferta cifrada $c = E(m)$ de otro participante puede construir $E(m + k) = c \cdot E(k)$ para hacer una oferta ligeramente superior sin conocer m .

Mitigación: Combinar el cifrado Paillier con esquemas de firma digital (Message Authentication Code - MAC) para garantizar la autenticidad e integridad del texto cifrado. Adicionalmente, usar Zero-Knowledge Proofs (ZKP) para demostrar que un texto cifrado es válido sin revelar su contenido.

8. Aplicaciones Prácticas

El Criptosistema de Paillier encuentra aplicaciones en sistemas reales donde se requiere operar sobre datos cifrados:

8.1 Votación Electrónica

Es la aplicación paradigmática de Paillier. Cada votante cifra su voto (0 o 1) con la clave pública del sistema. El administrador electoral multiplica todos los votos cifrados y descifra el resultado, obteniendo directamente el total de votos sin revelar voto individual alguno. La Universidad Católica de Lovaina usó un protocolo basado en Paillier en elecciones reales con más de 4,000 votos en 2010 (documentado en Grokipedia, Paillier cryptosystem).

8.2 Aprendizaje Federado Preservando la Privacidad

En sistemas de Federated Learning, múltiples clientes entrenan modelos de machine learning localmente y comparten gradientes cifrados con Paillier. El servidor agrega los gradientes cifrados (usando la propiedad homomórfica) sin acceder a los datos privados de cada cliente. Aplicaciones: modelos de salud distribuidos y sistemas de detección de

fraude bancario.

8.3 Computación Segura en la Nube

Servicios de cómputo en la nube pueden realizar operaciones de agregación y estadísticas (sumas, promedios) sobre datos cifrados del cliente. El cliente mantiene la clave privada y solo descifra el resultado final, garantizando que ni el proveedor de nube accede a los datos en claro.

8.4 Sistemas de Subastas Electrónicas

Paillier garantiza la confidencialidad de las ofertas durante la subasta, previniendo que subastadores deshonestos o coaliciones de participantes manipulen los resultados. La aplicación requiere combinarse con Zero-Knowledge Proofs para validar que las ofertas son valores válidos.

8.5 Transacciones Financieras de Privacidad

Se utiliza en protocolos de privacidad financiera para calcular sumas de transacciones o saldos agregados sin revelar montos individuales. También es componente de algunos protocolos de firma umbral ECDSA (Threshold ECDSA), documentados en Wikipedia (Paillier cryptosystem, sección de aplicaciones).

9. Análisis de Seguridad y Comparación

El Criptosistema de Paillier es un algoritmo altamente especializado, cuya seguridad y eficiencia presentan contrastes notables al compararse con esquemas clásicos como el de Rabin:

Fortalezas:

- Seguridad Semántica (IND-CPA): Es un cifrado probabilístico; gracias al uso de un factor aleatorio (r), el mismo mensaje genera un texto cifrado diferente en cada ejecución, lo que impide ataques de diccionario.
- Homomorfismo Aditivo: Es su mayor fortaleza técnica, permitiendo realizar sumas sobre datos cifrados sin necesidad de descifrarlos, garantizando la privacidad en procesos de cómputo delegado.

- Basado en un problema difícil (DCRA): Su seguridad descansa en la *Decisional Composite Residuosity Assumption*, un problema matemático considerado computacionalmente intratable sin conocer la factorización del módulo n .
- Resistencia a la factorización: Al igual que el sistema de Rabin, se beneficia de la dificultad de factorizar números compuestos grandes, proporcionando una base de seguridad sólida y bien estudiada.

Debilidades:

- Rendimiento y Carga Computacional: El proceso requiere operaciones de exponenciación modular pesadas, lo que lo hace significativamente más lento que otros algoritmos asimétricos.
- Expansión del Texto Cifrado: El cifrado resultante es aproximadamente el doble de grande (2x) que el módulo n , lo que genera un alto consumo de ancho de banda.
- Vulnerabilidad IND-CCA2 (Maleabilidad): Debido a su propiedad homomórfica, es inherentemente maleable; sin protecciones adicionales como el padding OAEP, es vulnerable a ataques de texto cifrado elegido.
- Sensibilidad Cuántica: Al depender de la factorización de enteros, es vulnerable al algoritmo de Shor, lo que comprometería su seguridad ante computadoras cuánticas potentes.

Comparación con otros algoritmos asimétricos:

- **Criptosistema de Rabin:** A diferencia de Paillier, la seguridad de Rabin es equivalente de manera comprobable a la factorización de enteros. Mientras Rabin es extremadamente rápido en el cifrado (solo requiere una elevación al cuadrado), Paillier es mucho más lento pero ofrece homomorfismo aditivo, algo que Rabin no posee.
- **RSA (con OAEP):** RSA es el estándar para confidencialidad general, pero carece de propiedades homomórficas; Paillier es superior únicamente cuando se requiere operar con los datos cifrados.
- **ElGamal:** Ambos presentan una expansión de texto de 2x; sin embargo, ElGamal ofrece homomorfismo multiplicativo, mientras que Paillier ofrece homomorfismo aditivo, orientándolos a usos distintos.

3. ElGamal

1.Introducción

El origen del algoritmo ElGamal responde a una necesidad fundamental de la criptografía moderna: permitir que dos partes intercambien información cifrada sin necesidad de compartir previamente una clave secreta. Esto lo diferencia radicalmente de algoritmos simétricos como AES o DES, donde emisor y receptor deben acordar una clave compartida antes de comunicarse, lo que plantea el llamado "problema de distribución de claves"(EcuRed, s.f.; SolidChain, 2024).

En 1984, el criptógrafo de origen egipcio Taher ElGamal propuso en su artículo "*A Public Key Cryptosystem and a Signature Scheme Based on Discrete Logarithms*" un nuevo sistema criptográfico basado en la dificultad computacional del problema del logaritmo discreto. Este problema consiste en que, dados tres números enteros g , p y A , resulta extremadamente difícil encontrar el valor x tal que $g^x \equiv A \pmod{p}$, aunque verificar la solución correcta es trivial.

La propuesta de ElGamal se apoya en el trabajo previo de Diffie y Hellman (1976), quienes introdujeron el concepto de criptografía de clave pública. Sin embargo, ElGamal extendió esas ideas para construir un sistema completo de cifrado y firma digital. Aunque ElGamal nunca fue proclamado estándar oficial del NIST, su influencia fue determinante: el DSA (Digital Signature Algorithm), adoptado como estándar federal en 1994, es una variante directa del esquema de firma de ElGamal (Whitfield Diffie & Martin Hellman, 1976).

2.Características Técnicas y Funcionamiento del Algoritmo ElGamal

El cifrado ElGamal se define técnicamente como un algoritmo de cifrado asimétrico (o de clave pública). A diferencia de AES que usa la misma clave para cifrar y descifrar, ElGamal utiliza un par de claves matemáticamente relacionadas: una clave pública, que cualquiera puede conocer, y una clave privada, que solo posee el destinatario legítimo (EcuRed, s.f.).

A.Fundamento Matemático

La seguridad de ElGamal descansa sobre la estructura algebraica de un **grupo cíclico**, típicamente el grupo multiplicativo de enteros módulo un número primo p (SolidChain, 2024).

Los elementos clave son:

- p : un número primo grande (2048 bits o más en implementaciones actuales).
- g : un generador del grupo, es decir, un elemento cuyas potencias sucesivas módulo p producen todos los elementos del grupo.
- x : la clave privada, un entero secreto elegido aleatoriamente.
- $A = g^x \bmod p$: la clave pública, derivada de la privada.

La asimetría surge de que calcular A a partir de x es computacionalmente sencillo, mientras que recuperar x a partir de A es computacionalmente inviable para valores grandes de p (Taher ElGamal, 1985).

B. Generación de Claves

Antes de poder cifrar o descifrar, el destinatario debe generar su par de claves (EcuRed, s.f.):

1. Se elige un número primo grande p y un generador g del grupo multiplicativo módulo p . Ambos son públicos.
2. Se selecciona un entero secreto x tal que $1 < x < p-1$. Esta es la **clave privada**.
3. Se calcula $A = g^x \bmod p$. Este valor, junto con p y g , constituye la **clave pública**.

El destinatario público (p, g, A) y guardar en secreto.

C. El Proceso de Cifrado

Para cifrar un mensaje M (representado como un entero tal que $1 < M < p-1$), el emisor sigue estos pasos (SolidChain, 2024):

1. **Obtiene la clave pública del destinatario:** (p, g, A) .

2. **Elige un número aleatorio efímero k** , tal que $1 < k < p-1$. Este valor k es único para cada mensaje y se descarta tras el cifrado.
3. **Calcula el primer componente del texto cifrado:** $C_1 = g^k \bmod p$
4. **Calcula el segundo componente del texto cifrado:** $C_2 = M \cdot A^k \bmod p$
5. **El texto cifrado** es el par (C_1, C_2) .

El valor $A^k \bmod p$ actúa como una "máscara aleatoria" que oculta el mensaje. Solo quien conozca x puede reconstruirla.

D.Proceso de Descifrado

El destinatario, en posesión de su clave privada x , recupera el mensaje original (Taher ElGamal, 1985):

1. A partir de C_1 y x , calcula la máscara: $S = C_1^x \bmod p$. Esto es posible porque $C_1^x = (g^k)^x = g^{kx} \bmod p$, que coincide con $A^k = (g^x)^k = g^{kx} \bmod p$
2. Calcula el inverso modular de S : $S^{-1} \bmod p$.
3. Recupera el mensaje original: $M = C_2 \cdot S^{-1} \bmod p$.

La correctitud se verifica porque: $C_2 \cdot S^{-1} = (M \cdot A^k) \cdot (A^k)^{-1} = M \pmod{p}$

3.Ventajas del Algoritmo ElGamal

- Es un sistema de clave pública, por lo que no necesita compartir previamente una clave secreta (EcuRed, s.f.).
- Su seguridad se basa en el problema del logaritmo discreto, que es matemáticamente difícil de resolver.
- Introduce aleatoriedad mediante el valor k , lo que hace que un mismo mensaje cifrado dos veces produzca resultados diferentes.
- Puede usarse también como base para esquemas de firma digital.

4.Desventajas del Algoritmo ElGamal

- El texto cifrado ocupa más espacio que el mensaje original, porque genera dos componentes (C_1, C_2) .

- Es más lento que algoritmos simétricos como AES.
- No suele usarse para cifrar grandes cantidades de datos directamente; normalmente se emplea en esquemas híbridos (SolidChain, 2024).

4. Criptosistema de Rabin

4.1. Definición y Origen

El Criptosistema de Rabin fue propuesto en 1979 por Michael O. Rabin en su artículo técnico *"Digitalized Signatures and Public-Key Functions as Intractable as Factorization"*, publicado por el MIT Laboratory for Computer Science. Se trata de un sistema de clave pública asimétrico cuya seguridad ha sido formalmente demostrada como computacionalmente equivalente al problema de factorización entera: romper el cifrado es tan difícil como factorizar el módulo público n . Esta equivalencia constituye una ventaja teórica significativa frente a RSA, cuya resistencia no ha sido reducida formalmente al mismo problema. (Rabin, 1979)

La criptografía de clave pública, introducida por Diffie y Hellman en 1976 (Diffie & Hellman, 1976), se fundamenta en la existencia de funciones de un solo sentido con trampa (*trapdoor one-way functions*). Rabin identificó la elevación al cuadrado modular como una de estas funciones y construyó sobre ella un criptosistema asimétrico con garantías de seguridad demostrables. (Boneh, 1999)

4.2. Fundamento Matemático

El sistema se basa en la elevación al cuadrado en aritmética modular. Dado un mensaje m y un

módulo $n = p \times q$ (producto de dos números primos grandes), el cifrado consiste en: (Rabin, 1979)

$$c \equiv m^2 \pmod{n}$$

Calcular la raíz cuadrada modular de c (el proceso de descifrado) es computacionalmente intratable sin conocer los factores p y q . Esta asimetría constituye el principio de puerta trasera (*trapdoor*) del sistema. Formalmente, se ha demostrado que cualquier algoritmo polinomial capaz de descifrar mensajes Rabin puede ser convertido en un algoritmo de factorización de igual complejidad. (Boneh, 1999); (Menezes et al., 1996)

El sistema involucra tres operaciones fundamentales:

- Generación de claves: elección de primos p y q con $p \equiv q \equiv 3 \pmod{4}$.
- Cifrado: cálculo de $c = m^2 \pmod{n}$ usando únicamente la clave pública n .
- Descifrado: cálculo de las 4 raíces cuadradas modulares de c usando (p, q) .

4.3. Residuos Cuadráticos y Teorema Chino del Resto

Un entero a se denomina residuo cuadrático módulo n si existe un entero x tal que $x^2 \equiv a \pmod{n}$. En el Criptosistema de Rabin, el texto cifrado c siempre es un residuo cuadrático módulo n , ya que resulta de elevar al cuadrado un valor entero. (Menezes et al., 1996)

$$x^2 \equiv a \pmod{n} \Rightarrow a \text{ es residuo cuadrático mod } n$$

El Teorema Chino del Resto (CRT), cuya formulación moderna se atribuye a trabajos recopilados en el siglo III d.C. pero cuya aplicación criptográfica fue formalizada por Menezes, van Oorschot y Vanstone (Menezes et al., 1996), permite combinar dos raíces parciales (una módulo p y otra módulo q) en cuatro raíces módulo n . Si $r^2 \equiv c \pmod{p}$ y $s^2 \equiv c \pmod{q}$, las cuatro raíces módulo n se obtienen mediante:

$$x_1 = (a + b) \pmod{n} \quad x_2 = n - x_1$$

$$x_3 = (a - b) \pmod{n} \quad x_4 = n - x_3$$

$$\text{donde: } a = r \cdot q \cdot (q^{-1} \pmod{p}) \pmod{n} \quad \text{y} \quad b = s \cdot p \cdot (p^{-1} \pmod{q}) \pmod{n}$$

Esta multiplicidad de cuatro soluciones es inherente al sistema y es la causa principal de la ambigüedad en el descifrado, que se resuelve mediante el uso de redundancia en el mensaje

original. [\(Menezes et al., 1996\)](#); [\(Paar & Pelzl, 2010\)](#)

4.4. Generación de Claves

La seguridad del Criptosistema de Rabin depende fundamentalmente de la correcta elección de los primos p y q . Estos deben cumplir una condición matemática específica —congruencia a 3 módulo 4— que permite aplicar la fórmula de Euler para el cálculo eficiente de raíces cuadradas modulares durante el descifrado. [\(Rabin, 1979\)](#); [\(Menezes et al., 1996\)](#)

Algoritmo de Generación de Claves

1. Elegir dos primos distintos suficientemente grandes p y q .
2. Verificar que $p \equiv 3 \pmod{4}$ y $q \equiv 3 \pmod{4}$.
3. Calcular $n = p \times q$ (módulo público).
4. Publicar n como clave pública. Mantener (p, q) en secreto como clave privada.

¿Por qué $p \equiv 3 \pmod{4}$?

Cuando $p \equiv 3 \pmod{4}$, la raíz cuadrada de c módulo p se calcula directamente usando el Lema de Euler (también denominado Criterio de Euler):

$$r = c^{(p+1)/4} \pmod{p}$$

Esta fórmula tiene complejidad $O(\log p)$ mediante exponenciación rápida (square-and-multiply). Sin esta condición, sería necesario recurrir al algoritmo de Tonelli-Shanks, de mayor complejidad. Primos válidos (≤ 100): 3, 7, 11, 19, 23, 31, 43, 47, 59, 67, 71, 79, 83. [\(Menezes et al., 1996\)](#)

Ejemplo de Generación de Claves

Parámetro	Valor	Verificación

Primo p	7	$7 \bmod 4 = 3$
Primo q	11	$11 \bmod 4 = 3$
Módulo $n = p \times q$	77	$7 \times 11 = 77$
Clave pública	$n = 77$	Se publica abiertamente
Clave privada	$(p=7, q=11)$	Solo el receptor la conoce

En implementaciones reales orientadas a seguridad, los primos deben tener al menos 1024 bits cada uno, obteniendo un módulo n de 2048 bits o más, conforme a las recomendaciones del NIST ([NIST, 2020](#)). Se generan con generadores de números aleatorios criptográficamente seguros (CSPRNG) y su primalidad se verifica mediante la prueba probabilística de Miller-Rabin con suficientes rondas para garantizar una probabilidad de error inferior a 2^{-128} . ([Paar & Pelzl, 2010](#))

4.5. Proceso de Cifrado

4.5.1. Algoritmo de Cifrado

El cifrado en el Criptosistema de Rabin requiere únicamente la clave pública n y consiste en una sola operación de cuadrado modular, lo que lo convierte en uno de los algoritmos asimétricos más eficientes en términos computacionales: ([Rabin, 1979](#)); ([Menezes et al., 1996](#))

$$c = m^2 \bmod n$$

Donde m es el bloque numérico del mensaje con $0 \leq m < n$. Para mensajes largos, el texto se divide en bloques y se aplica el cifrado a cada bloque de forma independiente. La operación $c = m^2 \bmod n$ tiene complejidad $O((\log n)^2)$ mediante el algoritmo de multiplicación modular estándar, considerablemente más eficiente que RSA, que requiere exponentes de mayor tamaño. ([Paar & Pelzl, 2010](#))

4.5.2. Ejemplo

Ejemplo: Mensaje 'ingenieria de sistemas'

Paso 1 — Conversión del mensaje a valores numéricos

Con $n = 77$, se utiliza la codificación: $a=1, b=2, \dots, z=26$, espacio=0. Todos los valores resultantes son estrictamente menores que 77, condición necesaria para que la operación modular sea válida.

Carácter	Valor	Carácter	Valor	Carácter	Valor	Carácter	Valor
i	9	g	7	i	9	s	19
n	14	e	5	e	5	t	20
g	7	r	18	e	5	m	13
e	5	i	9	a	1	s	19
n	14	a	1	(esp)	0		
i	9	d	4	d	4		
e	5	e	5	(esp)	0		
r	18	(esp)	0	s	19		

Secuencia numérica completa del mensaje:

$m = [9, 14, 7, 5, 14, 9, 5, 18, 9, 1, 0, 4, 5, 0, 19, 9, 19, 20, 5, 13, 1, 19]$

Paso 2 — Aplicación de $c = m^2 \bmod 77$

Carácter	m	m^2	$c = m^2 \bmod 77$
i	9	81	4
n	14	196	42
g	7	49	49
e	5	25	25
n	14	196	42
i	9	81	4
e	5	25	25
r	18	324	16
i	9	81	4

a	1	1	1
(esp)	0	0	0
d	4	16	16
e	5	25	25
(esp)	0	0	0
s	19	361	53
i	9	81	4
s	19	361	53
t	20	400	15
e	5	25	25
m	13	169	15
a	1	1	1
s	19	361	53

Mensaje cifrado resultante:

$c = [4, 42, 49, 25, 42, 4, 25, 16, 4, 1, 0, 16, 25, 0, 53, 4, 53, 15, 25, 15, 1, 53]$

4.6. Proceso de Descifrado**4.6.1. Algoritmo de Descifrado**

El descifrado requiere la clave privada (p, q) y sigue tres etapas fundamentales. La existencia de cuatro raíces cuadradas modulares es una consecuencia directa del Teorema Chino del Resto aplicado a módulos compuestos. Para determinar unívocamente el mensaje original, se requiere que el remitente haya agregado información de redundancia reconocible antes del cifrado. ([Rabin, 1979](#)); ([Menezes et al., 1996](#))

1. Calcular raíces parciales módulo p y q usando la fórmula de Euler.
2. Combinar las raíces con el Teorema Chino del Resto para obtener 4 candidatos módulo n.
3. Seleccionar el candidato correcto mediante el mecanismo de redundancia o contexto semántico.

Fórmulas de las Raíces Parciales

$$r = c^{\frac{p+1}{4}} \bmod p \quad s = c^{\frac{q+1}{4}} \bmod q$$

Para los primos del ejemplo: $p = 7 \rightarrow \text{exponente} = (7+1)/4 = 2$; $q = 11 \rightarrow \text{exponente} = (11+1)/4 = 3$. Estas fórmulas son válidas gracias a la condición $p \equiv q \equiv 3 \pmod{4}$ establecida en la generación de claves, tal como formalizó Rabin ([Rabin, 1979](#)) y se desarrolla en el Handbook of Applied Cryptography. ([Menezes et al., 1996](#))

4.6.2. Ejemplo

Ejemplo Paso a Paso — letra 'i' (m=9, c=4)

Partimos de $c = 4$, $p = 7$, $q = 11$, $n = 77$ ($9^2 \bmod 77 = 81 \bmod 77 = 4$).

Etapas 1 — Raíz mod $p = 7$

$$r = 4^{((7+1)/4)} \bmod 7 = 4^2 \bmod 7 = 16 \bmod 7 = 2$$

Etapas 2 — Raíz mod $q = 11$

$$s = 4^{((11+1)/4)} \bmod 11 = 4^3 \bmod 11 = 64 \bmod 11 = 9$$

Etapas 3 — Algoritmo Extendido de Euclides y CRT

$$11 \cdot y_q \equiv 1 \pmod{7} \rightarrow y_q = 2$$

$$7 \cdot y_p \equiv 1 \pmod{11} \rightarrow y_p = 8$$

$$a = r \cdot q \cdot y_q \bmod n = 2 \cdot 11 \cdot 2 \bmod 77 = 44$$

$$b = s \cdot p \cdot y_p \bmod n = 9 \cdot 7 \cdot 8 \bmod 77 = 504 \bmod 77 = 42$$

$$x_1 = (44 + 42) \bmod 77 = 86 \bmod 77 = 9 \leftarrow \text{mensaje original}$$

$$x_2 = 77 - 9 = 68$$

$$x_3 = (44 - 42 + 77) \bmod 77 = 79 \bmod 77 = 2$$

$$x_4 = 77 - 2 = 75$$

Las cuatro raíces de $c = 4$ módulo 77 son:

Raíz	Valor	Letra	¿Es el mensaje?
x_1	9	i	Sí — coincide con el patrón de

			redundancia
x_2	68	—	No — fuera del rango a-z (1-26)
x_3	2	b	No — no coincide con la redundancia
x_4	75	—	No — fuera del rango a-z (1-26)

4.6.3. Tabla de Descifrado Completo

A continuación, se muestra el proceso de descifrado para cada carácter del mensaje 'ingeniería de sistemas'. En cada caso, las cuatro raíces se calculan mediante el procedimiento anterior y el mensaje correcto se identifica por el patrón de redundancia o por estar en el rango válido de codificación (1-26 para letras, 0 para espacio). ([Menezes et al., 1996](#))

Original	c (cifrado)	4 raíces posibles	m correcto	Recuperado
i	4	9, 68, 2, 75	9	i
n	42	14, 63, 63, 14	14	n
g	49	70, 7, 7, 70	7	g
e	25	16, 61, 72, 5	5	e
n	42	14, 63, 63, 14	14	n

i	4	9, 68, 2, 75	9	i
e	25	16, 61, 72, 5	5	e
r	16	4, 73, 18, 59	18	r
i	4	9, 68, 2, 75	9	i
a	1	1, 76, 43, 34	1	a
(esp)	0	0, 0, 0, 0	0	(esp)
d	16	4, 73, 18, 59	4	d
e	25	16, 61, 72, 5	5	e
(esp)	0	0, 0, 0, 0	0	(esp)
s	53	58, 19, 30, 47	19	s
i	4	9, 68, 2, 75	9	i
s	53	58, 19, 30, 47	19	s
t	15	64, 13, 57, 20	20	t

e	25	16, 61, 72, 5	5	e
m	15	64, 13, 57, 20	13	m
a	1	1, 76, 43, 34	1	a
s	53	58, 19, 30, 47	19	s

Mensaje descifrado: "ingenieria de sistemas"

4.7. Análisis de Seguridad

El análisis de seguridad del Criptosistema de Rabin comprende la evaluación de sus propiedades formales de resistencia, sus limitaciones prácticas y los vectores de ataque conocidos. A diferencia de RSA, Rabin posee una prueba de seguridad formal equivalente al problema de factorización, lo que le otorga un estatus teórico privilegiado en la criptografía de clave pública. ([Rabin, 1979](#)); ([Boneh, 1999](#)); ([Stinson & Paterson, 2018](#))

4.7.1. Ventajas del Criptosistema de Rabin

Seguridad Demostrada Formalmente. Es el único criptosistema de clave pública cuya seguridad ha sido reducida formalmente al problema de factorización entera en el modelo de seguridad estándar. Rabin demostró en 1979 que cualquier algoritmo polinomial capaz de descifrar mensajes puede transformarse en un algoritmo de factorización de igual eficiencia. ([Rabin, 1979](#)) RSA no goza de esta reducción formal. ([Stinson & Paterson, 2018](#))

Cifrado Extremadamente Eficiente. El cifrado requiere solo una multiplicación modular ($c = m^2 \bmod n$), una operación de complejidad $O((\log n)^2)$. Esto lo hace

significativamente más rápido que RSA, que requiere exponentes de varios cientos de bits para el cifrado. (Menezes et al., 1996) Es especialmente adecuado para dispositivos con recursos limitados como tarjetas inteligentes y sistemas embebidos. (Paar & Pelzl, 2010)

Simplicidad de Implementación. El algoritmo es conceptualmente simple y sus operaciones primitivas están bien estudiadas. La generación de claves solo requiere dos primos con una condición de congruencia adicional, lo que reduce la superficie de ataque derivada de errores de implementación. (Menezes et al., 1996)

Base Teórica Sólida. El problema de raíces cuadradas modulares sobre módulos compuestos es uno de los problemas más estudiados en teoría de números computacional. Su equivalencia computacional con la factorización entera ha sido verificada en múltiples trabajos independientes. (Boneh, 1999); (Stinson & Paterson, 2018)

Aplicabilidad en Firmas Digitales. El esquema original de Rabin incluyó un mecanismo de firma digital donde la dificultad de producir firmas falsas se reduce formalmente a la dificultad de factorizar n . Esto garantiza seguridad en esquemas de no repudio bajo supuestos estándar. (Rabin, 1979)

4.7.2. Desventajas del Criptosistema de Rabin

Ambigüedad en el Descifrado. Todo descifrado produce cuatro raíces cuadradas modulares posibles. Sin información adicional (redundancia o padding), es imposible determinar unívocamente cuál corresponde al mensaje original. Esta propiedad no ocurre en RSA con exponente público fijo y requiere protocolos adicionales. (Menezes et al., 1996)

Vulnerabilidad a Ataques de Texto Cifrado Elegido (CCA). En su forma básica, el sistema es rompible con un ataque CCA adaptativo. Un adversario con acceso a un oráculo de descifrado puede recuperar la factorización de n con $O(1)$ consultas, con probabilidad $1/2$ por intento. (Goldwasser & Micali, 1984) Requiere combinarse con un esquema de padding como OAEP para lograr seguridad CCA2. (Bellare & Rogaway, 1994)

Asimetría Computacional en el Descifrado. El descifrado requiere exponenciaciones modulares, cálculo de inversos mediante el algoritmo extendido de Euclides y aplicación del CRT, lo que genera una asimetría computacional notable respecto al cifrado. Aunque cada operación es polinomial, la constante es mayor que en el cifrado. (Paar & Pelzl, 2010)

Redundancia Obligatoria. Para hacer el descifrado determinístico, es necesario agregar redundancia al mensaje antes de cifrar, incrementando el tamaño del texto cifrado y agregando complejidad al protocolo de comunicación. (Menezes et al., 1996)

Menor Adopción Industrial. A pesar de sus ventajas teóricas, Rabin tiene menor adopción que RSA o ECC. La mayoría de las bibliotecas criptográficas estándar (OpenSSL, Bouncy Castle, libsodium) no lo implementan directamente, limitando su interoperabilidad en entornos de producción. (Paar & Pelzl, 2010)

4.7.3. Modelos de Ataque

A continuación, se describen los principales vectores de ataque conocidos contra el Criptosistema de Rabin, su viabilidad práctica y las contramedidas recomendadas por la literatura especializada. (Menezes et al., 1996); (Stinson & Paterson, 2018)

4.7.3.1. Factorización de n

Es el ataque más directo al sistema. Si un adversario factoriza $n = p \times q$, puede reconstruir la clave privada y descifrar cualquier mensaje. El algoritmo más eficiente conocido en computación clásica es la Criba del Cuerpo de Números General (GNFS), con complejidad subexponencial $L_n[1/3, (64/9)^{(1/3)}]$. (Paar & Pelzl, 2010) En computación cuántica, el algoritmo de Shor (Shor, 1994) factoriza en tiempo polinomial $O((\log n)^3)$, lo que representa una amenaza a largo plazo. Contramedida: usar módulos n de al menos 2048 bits conforme a NIST SP 800-57 (NIST, 2020); evaluar migración a esquemas post-cuánticos para sistemas con horizonte de seguridad superior a 10 años.

4.7.3.2. Ataque de Texto Cifrado Elegido (CCA)

Es el ataque más severo contra el Criptosistema de Rabin básico, demostrado formalmente por Rabin en 1979 y analizado en detalle por Menezes et al. El adversario: (1) selecciona r aleatorio con $0 < r < n$; (2) construye $c^* = r^2 \bmod n$; (3) solicita al oráculo

descifrar c^* ; (4) con probabilidad $1/2$, la respuesta x satisface $x \neq \pm r \pmod{n}$, de donde $\gcd(r - x, n) \in \{p, q\}$. (Rabin, 1979); (Menezes et al., 1996) Contramedida: nunca exponer un oráculo de descifrado no autenticado; aplicar Rabin-OAEP para obtener seguridad CCA2 demostrable. (Bellare & Rogaway, 1994)

4.7.3.3. Primos Débiles

Si los primos p y q son cercanos entre sí ($|p - q|$ pequeño), el método de factorización de Fermat puede descomponer n eficientemente en tiempo $O(n^{1/4})$ o menos. Si los primos son generados con un CSPRNG débil o predecible, pueden ser inferidos por análisis estadístico. (Menezes et al., 1996); (Stinson & Paterson, 2018) Contramedidas: verificar que $|p - q| > 2^{n/4}$; generar primos utilizando generadores de números aleatorios criptográficamente seguros certificados (e.g., AES-CTR-DRBG conforme a NIST SP 800-90A (NIST, 2020)); aplicar pruebas de primalidad con cota de error $\leq 2^{-128}$.

5. Merkle-Hellman

5.1 Introducción y Contexto Histórico

El algoritmo de Merkle-Hellman, propuesto por Ralph Merkle y Martin Hellman en 1978, fue uno de los primeros criptosistemas de clave pública en desarrollarse (Merkle & Hellman, 1978). Su seguridad se fundamenta teóricamente en un problema matemático clasificado como NP-completo: el Problema de la Suma de Subconjuntos, comúnmente conocido como el "Problema de la Mochila" (Schneier, 1996). La premisa consiste en un conjunto de elementos con pesos distintos y una capacidad objetivo, donde el reto radica en determinar qué combinación exacta de elementos suma el peso total objetivo; resolver una mochila general es computacionalmente inviable, mientras que resolver una estructurada bajo reglas específicas es trivial (Stallings, 2017).

5.1.1 Generación de Claves

Para que la comunicación sea segura, el receptor debe generar un par de claves matemática y lógicamente vinculadas mediante el uso de parámetros modulares (Katz & Lindell, 2014).

5.1.1.1 Creación de la Clave Privada (La Mochila Super Creciente)

Se debe definir una secuencia de números W donde cada elemento sea estrictamente mayor a la suma de todos los elementos que lo preceden, lo cual asegura que el descifrado posterior

tenga una única solución posible y evita colisiones de datos (Merkle & Hellman, 1978). Sea nuestra secuencia privada de 4 bits:

$$W = \{2, 3, 6, 13\}$$

(Comprobación de la regla supercreciente: $3 > 2$; $6 > 3 + 2$; $13 > 6 + 3 + 2$)

5.1.1.2 Definición de Parámetros Modulares

Se requieren dos valores matemáticos para ocultar la secuencia original mediante aritmética modular (Katz & Lindell, 2014):

1. Módulo (q): Debe ser estrictamente mayor a la suma de todos los elementos de W ($2+3+6+13 = 24$). Elegimos $q = 29$.
2. Multiplicador (r): Debe ser un número entero coprimo con q (es decir $\{MCD\}(r, q) = 1$). Elegimos $r = 11$.

5.1.1.3 Creación de la Clave Pública

La clave pública B se genera multiplicando cada elemento de la clave privada W por el multiplicador r , aplicando el módulo q para enmascarar los valores originales (Schneier, 1996). La fórmula aplicada es:

$$b_i = (w_i \cdot r) \pmod{q}$$

Calculando cada elemento:

- $b_1 = (2 \cdot 11) \pmod{29} = 22$
- $b_2 = (3 \cdot 11) \pmod{29} = 33 \pmod{29} = 4$
- $b_3 = (6 \cdot 11) \pmod{29} = 66 \pmod{29} = 8$
- $b_4 = (13 \cdot 11) \pmod{29} = 143 \pmod{29} = 27$

La **Clave Pública** resultante a compartir es: $B = \{22, 4, 8, 27\}$.

5.2 Proceso de Cifrado

El emisor utiliza la Clave Pública B para ocultar su mensaje sumando los pesos correspondientes a los bits activos del texto plano (Stallings, 2017). Si deseamos enviar el mensaje binario de 4 bits **1011**, cada bit del mensaje se multiplica por su peso correspondiente en la clave pública, sumando los resultados:

$$C = \sum (M_i \cdot b_i)$$

$$C = (1 \cdot 22) + (0 \cdot 4) + (1 \cdot 8) + (1 \cdot 27)$$

$$C = 22 + 0 + 8 + 27 = 57$$

El Texto Cifrado (C) que se transmite a través del canal inseguro es el número **57**.

5.3 Proceso de Descifrado

El receptor, al poseer los parámetros privados originales (r y q), puede revertir el cifrado mediante aritmética modular.

5.3.1 Cálculo del Inverso Modular

Se debe encontrar un valor r^{-1} que anule el multiplicador original, cumpliendo la condición:

$$(r \cdot r^{-1}) \pmod{q} = 1$$

$$(11 \cdot r^{-1}) \pmod{29} = 1$$

Resolviendo, el inverso modular es $r^{-1} = 8$ (ya que $11 \cdot 8 = 88$, y $88 \pmod{29} = 1$).

5.3.2 Transformación del Texto Cifrado

Se multiplica el criptograma interceptado por el inverso modular para obtener un "mensaje limpio" (C'):

$$C' = (C \cdot r^{-1}) \pmod{q}$$

$$C' = (57 \cdot 8) \pmod{29} = 456 \pmod{29} = 21$$

5.3.3 Resolución de la Mochila (Método Voraz)

Con el valor recuperado de **21**, el receptor utiliza su Clave Privada super creciente $W = \{2, 3, 6, 13\}$ para determinar qué elementos suman exactamente esta cantidad, evaluando de mayor a menor (Schneier, 1996):

- ¿ $13 \leq 21$? **Sí**. (Bit = 1). Restante: $21 - 13 = 8$
- ¿ $6 \leq 8$? **Sí**. (Bit = 1). Restante: $8 - 6 = 2$
- ¿ $3 \leq 2$? **No**. (Bit = 0). Restante: 2
- ¿ $2 \leq 2$? **Sí**. (Bit = 1). Restante: $2 - 2 = 0$

El mensaje descifrado se lee reconstruyendo los bits: **1011**. El proceso es exitoso y matemáticamente exacto.

5.4 Ventajas, Desventajas y Vulnerabilidades

5.4.1 Ventajas:

- Alta eficiencia computacional en las operaciones de cifrado y descifrado (requiere únicamente sumas y multiplicaciones simples), comparado con algoritmos que requieren exponenciación como RSA.
- Pionero en la aplicación de problemas NP-Complejos en la criptografía asimétrica.

5.4.2 Desventajas y Vulnerabilidad (Ataque de Shamir):

- Requiere claves públicas de gran tamaño para intentar ser seguro frente a la fuerza bruta, lo que aumenta la carga de almacenamiento.
- La vulnerabilidad crítica del sistema fue demostrada en 1984, cuando el criptógrafo Adi Shamir rompió el algoritmo de manera definitiva (Shamir, 1984). Shamir demostró que la clave pública, al ser generada a partir de una secuencia super creciente, preserva

ciertas estructuras matemáticas subyacentes. Utilizando técnicas de reducción de bases de retículos (como el algoritmo LLL), demostró que un atacante puede deducir la clave privada a partir de la pública en tiempo polinómico, haciendo que este algoritmo sea completamente obsoleto e inseguro para aplicaciones modernas (Shamir, 1984).

BIBLIOGRAFÍA

- Katz, J., & Lindell, Y. (2014): Introduction to Modern Cryptography. CRC Press. (Libro de texto fundamental en ingeniería para entender el cambio de DES a AES).
- <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197.pdf>
- <https://youtu.be/tzj1RoqRnv0>
- <https://www.springer.com/gp/book/9783540420613>
- Bernstein, D. J. (2008). *ChaCha, a variant of Salsa20*. Workshop Record of SASC 2008: The State of the Art of Stream Ciphers.(Documento original donde se presenta e introduce el algoritmo ChaCha20.)
- Nir, Y., & Langley, A. (2018). *ChaCha20 and Poly1305 for IETF Protocols. RFC 8439. Internet Engineering Task Force (IETF)*.(Especificación oficial utilizada para la explicación del funcionamiento del algoritmo,nonce, contador, generación del keystream y estructura interna.)
- Menezes, A. J., Van Oorschot, P. C., & Vanstone, S. A. (1996). *Handbook of Applied Cryptography*. CRC Press. (Conceptos matemáticos generales utilizados para explicar operaciones XOR y modelos de cifrado simétrico.)
- Merkle, R., & Hellman, M. (1978). *Hiding information and signatures in trapdoor knapsacks*. IEEE Transactions on Information Theory, 24(5), 525-530. (Este es el documento original donde se inventó el algoritmo).
- Shamir, A. (1984). *A polynomial-time algorithm for breaking the basic Merkle-Hellman cryptosystem*. IEEE Transactions on Information Theory, 30(5), 699-704. (Este es el documento histórico donde Adi Shamir demuestra cómo romper el cifrado).
- Wikipedia.PaillierCryptosystem.https://en.wikipedia.org/wiki/Paillier_cryptosystem
- Wikipedia (ES). Sistema criptográfico Paillier.
- Stallings, W. (2017). *Fundamentos de Seguridad en Redes: Aplicaciones y Estándares* (6ta ed.). Pearson Educación. (Excelente referencia general para tu materia de ciberseguridad).

- Fuente web — EcuRed EcuRed. (s.f.). *ElGamal*. Recuperado el 5 de abril de 2026, de <https://www.ecured.cu/ElGamal>
- Fuente web — SolidChain SolidChain. (2024, 31 de diciembre). *ElGamal*. Recuperado el 5 de abril de 2026, de <https://solidchain.ar/index.php/2024/12/31/elgamal/>
- Fuente audiovisual — YouTube [Autor del video]. (s.f.). *ElGamal* [Video]. YouTube. <https://www.youtube.com/watch?v=N3ANaOi9gAc>
- Fuente primaria — Artículo original ElGamal, T. (1985). *A public key cryptosystem and a signature scheme based on discrete logarithms*. IEEE Transactions on Information Theory, 31(4), 469–472.
- Referencia complementaria — Diffie & Hellman Diffie, W., & Hellman, M. (1976). *New directions in cryptography*. IEEE Transactions on Information Theory, 22(6), 644–654.
- <https://www.geeksforgeeks.org/computer-networks/rsa-algorithm-cryptography/>
- [https://en.wikipedia.org/wiki/RSA_\(cryptosystem\)](https://en.wikipedia.org/wiki/RSA_(cryptosystem))
- <https://www.techtarget.com/searchsecurity/definition/RSA>
- https://www.splunk.com/en_us/blog/learn/rsa-algorithm-cryptography.html
- <https://www.encryptionconsulting.com/education-center/what-is-rsa/>
- <https://securew2.com/blog/what-is-rsa-asymmetric-encryption>
- <https://www.cs.sjsu.edu/faculty/stamp/students/article.html>
- https://en.wikipedia.org/wiki/Timing_attack
- <https://koofr.eu/blog/posts/xsalsa20-encryption-vs-aes-256-whats-the-difference>
- <https://en.wikipedia.org/wiki/Salsa20>
- Rabin, M. O. (1979). Digitalized Signatures and Public-Key Functions as Intractable as Factorization. MIT Laboratory for Computer Science, Technical Report MIT/LCS/TR-212. Massachusetts Institute of Technology.
- Diffie, W., & Hellman, M. E. (1976). New Directions in Cryptography. IEEE Transactions on Information Theory, 22(6), 644–654. <https://doi.org/10.1109/TIT.1976.1055638>
- Boneh, D. (1999). Twenty Years of Attacks on the RSA Cryptosystem. Notices of the American Mathematical Society (AMS), 46(2), 203–213.
- Menezes, A. J., van Oorschot, P. C., & Vanstone, S. A. (1996). Handbook of Applied Cryptography. CRC Press. Disponible en: <https://cacr.uwaterloo.ca/hac/>
- Paar, C., & Pelzl, J. (2010). Understanding Cryptography: A Textbook for Students and Practitioners. Springer. <https://doi.org/10.1007/978-3-642-04101-3>
- National Institute of Standards and Technology — NIST. (2020). NIST Special

Publication 800-57 Part 1 Rev. 5: Recommendation for Key Management. U.S. Department of Commerce. <https://doi.org/10.6028/NIST.SP.800-57pt1r5>

- Stinson, D. R., & Paterson, M. B. (2018). *Cryptography: Theory and Practice* (4th ed.). Chapman and Hall/CRC Press.
- Goldwasser, S., & Micali, S. (1984). Probabilistic Encryption. *Journal of Computer and System Sciences*, 28(2), 270–299. [https://doi.org/10.1016/0022-0000\(84\)90070-9](https://doi.org/10.1016/0022-0000(84)90070-9)
- Bellare, M., & Rogaway, P. (1994). Optimal Asymmetric Encryption — How to Encrypt with RSA. In *Advances in Cryptology — EUROCRYPT 1994*. Lecture Notes in Computer Science, vol. 950. Springer. <https://doi.org/10.1007/BFb0053428>
- Shor, P. W. (1994). Algorithms for Quantum Computation: Discrete Logarithms and Factoring. *Proceedings of the 35th Annual Symposium on Foundations of Computer Science (FOCS)*. IEEE. <https://doi.org/10.1109/SFCS.1994.365700>
- Kocher, P., Jaffe, J., & Jun, B. (1999). Differential Power Analysis. In *Advances in Cryptology — CRYPTO 1999*. Lecture Notes in Computer Science, vol. 1666. Springer. https://doi.org/10.1007/3-540-48405-1_25

●

●